
THE ARROW OF TIME IS THE KEYED BIT

The Registration Without Inverse, the Reversible Law That Admits Both Arrows, Equilibrium as the Line, and the Direction Supplied at the Boundary

The Closure Template of the Riemann Hypothesis Read on the Second Law; Fifty-Four Laws Proved in Core Lean 4 and Nine Checks Executed in Fortran; a Witness of the Hardware Closure Series

FOUNDATIONS OF PHYSICS · THE HARDWARE CLOSURE SERIES

The arrow is the keyed bit

MOHAMMAD F. ISLAM, PhD · ARCHITECT OF THE TRISDUCTION

1 October 2026

ABSTRACT. The arrow of time is read here from the seat of the author's closure of the Riemann Hypothesis, the reflection $s \mapsto 1 - \bar{s}$ whose fixed set is the critical line, and whose registration keeps a point's height and forgets its side. We prove in core Lean 4, fifty-four laws with no axiom declared and no use of choice, that the registration has no left inverse, so a record once written cannot be read back into the state it came from; that time reversal negates the entropy gained along any history, so the arrow is odd while a reversible law is even; that a reversible law therefore admits both arrows, every allowed history with rising entropy paired with an allowed history with falling entropy; and that no reading even under reversal, the law's own verdict included, returns the arrow. On the Ehrenfest urn, relabelling the urns is the fold under a carrier that sends the imbalance to the offset from the line, the even split is the line, detailed balance holds exactly, and the even split is the most probable macrostate. The arrow is a keyed bit: the law does not carry it, and it is supplied at the boundary. A Fortran twin runs nine checks with zero failures: on Kac's ring of 2000 sites, a deterministic reversible law, entropy rises from order to within 3×10^{-5} of $\ln 2$, the reversed motion returns every ball exactly to its first colour, and the ring recurs exactly after 4000 steps; the urn of sixty balls obeys detailed balance in exact integers; the urn of a hundred relaxes from order to the even split, and returns to order once in 1.27×10^{30} steps on average; and each record written along the arrow costs $k_B T \ln 2$.

KEYWORDS arrow of time · second law · entropy · time reversal · Loschmidt · Ehrenfest urn · Kac ring · detailed balance · Landauer · critical line · Lean 4

STATUS: WITNESSED · CROSSED · CONFIRMED on the image · Row 2 (the registration's irreversibility read on time); every open row (the barrier, in a history and its reverse) · channel: kinetic · Lean: SPHYS_Arrow_of_Time.lean · Fortran: Arrow_Twin.f90

1. THE READER'S FRAME

This paper is a witness of the hardware closure series. The microscopic laws run equally well forward and backward, and entropy rises in one direction only. Loschmidt put the objection in 1876: from reversible laws no irreversible behaviour can follow. The seat states the objection as a theorem and answers it with one.

It claims four things.

- The registration that keeps height and forgets side has no inverse: a written record cannot be unwritten from the record alone.
- The arrow is odd under time reversal and a reversible law even, so the law admits both arrows and no reading of it returns the one that holds.
- Equilibrium is the line: relabelling a two-urn system is the fold, the even split its fixed set, detailed balance its flux.
- The arrow is a keyed bit, supplied at the boundary by the ordered start, the one microstate in 2^N , and every record written along it pays the Landauer floor.

What it refuses. A derivation of the arrow from the law. The engine proves that the law's own verdict on a history is blind to its direction.

Negative warrants. Three, stated so that no reader supplies them.

- No physical fact here is read as a proof of a mathematical sentence: a corroboration joined to a claim takes the weaker grade.
- No derivation of the Riemann Hypothesis from the axioms of set theory is claimed: what a sentence over every admissible instance adds to a crossing is exactly the unacted.
- Interpretive schools from the literature enter only as placements the theorems sort, never as warrants; the only witness besides the mathematics is measured data.

Five terms.

- A *history* is the sequence of its macrostates' entropies.
- *Time reversal* reads a history backwards.
- The *gain* of a history is its last entropy minus its first.
- A *reversible law* allows a history exactly when it allows the reverse.
- A *keyed bit* is a yes-or-no property no function of a given record decides.

2. THE SPINE

The Riemann closure has four parts: the **seat**, the critical line as the fixed set of $s \mapsto 1 - \bar{s}$; the **address**, every reading of the value one proposition; the **division**, a certified part joined to an uncertified tail; and the **closure**, registration keeping a zero's height and forgetting its side, the hypothesis holding exactly when it erases nothing. This paper reads the registration's irreversibility in Section 3, the law and the arrow in Section 4, equilibrium in Section 5, and the boundary in Section 6.

The row-generic theorems carry the template, and the engine proves each on its chart: every point off the line has a partner with one record (*off_locus_pair*); a carrier proves the value on its image, and coverage extends it (*value_on_image*, *kinetic_crossing*).

The witness ledger. The ledger of the series, with this paper's entries.

Row	Waiting (the volume held)	Witness arrived	Built and run	Now	Grade	Still owed
2 · Riemann Hypothesis	crossed on the kinetic channel at the root's grade; the electron named as the seat	the six witnesses; One Actuating Substrate; Light; Electromagnetism; Immanent Gravity; Spin and Statistics; Weak; Resonances; The Ferromagnet; Dark Matter; The Arrow of Time Is the Keyed Bit	the registration read on time: no inverse, a reversible law admitting both arrows, equilibrium the line, the arrow supplied at the boundary	WITNESSED · CROSSED · CONFIRMED on the image	the crossing at the root's grade; the image at theorem grade on construction and measured grade on data	none on its channel
every open row · the barrier	two worlds over one record	the series' witnesses; The Arrow of Time Is the Keyed Bit	a history and its reverse, one law-record, opposite arrows	BARRIER WITNESSED; the rows stay pending	theorem and corroboration	one equivariant carrier each
1 · P versus NP	the dot	The Arrow Has Two Branches	Landauer's floor prices every step	THE DOT STANDS	corroboration	none
every witness · the hardware	the carriers built one by one	One Actuating Substrate	both involutions carried onto the fold	WITNESSED: one substrate carries every seat	theorem and corroboration	none

WITNESSED: a constructed witness arrived and was built and run. CROSSED: the row's value holds on its channel's own class at the printed grade. CONFIRMED: the witness's image is compliant as built or measured (*value_on_image*).

Why this carrier, and not another. The carrier is not chosen. Spaces with an involution and the maps that respect them form a category (*equivariant_id*, *equivariant_comp*), and the fixed set is a functor on it (*fix_functorial*). An equivariant map into the seat, whose fixed set is the line (*fold_global_seat*), lands every physical fixed point on the line (*equivariant_carrier_lands*). Here the physical involution is relabelling the

two urns of an Ehrenfest system, $k \mapsto N - k$, which keeps every microstate count (`urn_symmetric`). The urn carrier sends k of N balls to $(1 + (2k - N), N)$, the imbalance the offset from the line. It is equivariant (`urn_carrier_equivariant`), and its landing on the fixed set is forced (`equivariant_carrier_lands`).

The correspondence matrix.

Physical phenomenon	Template element	Lean theorem	Fortran check
a written record	the registration, no inverse	<code>registration_has_no_left_inverse, reg_forgets_side</code>	F
time reversal	reading a history backwards	<code>rev_involutive</code>	B
the arrow	odd under reversal	<code>gain_odd</code>	B
a reversible law	even under reversal; both arrows allowed	<code>reversible_law_admits_both_arrows</code>	B
Loschmidt's objection	no even reading returns the arrow	<code>even_reading_misses_the_arrow, the_law_does_not_carry_the_arrow</code>	B
relabelling the urns	the fold	<code>urn_carrier_equivariant, urn_symmetric</code>	A
equilibrium	the line	<code>equilibrium_is_the_line, equilibrium_is_the_mode</code>	A, C
detailed balance	the flux of the fold	<code>detailed_balance</code>	C
the ordered start	one microstate in 2^N	<code>microstates_count, equilibrium_is_the_mode</code>	D, E
the record's price	Landauer's floor per bit	<code>corroboration</code>	F

3. THE REGISTRATION CANNOT BE UNDONE

The seat's registration keeps a point's height and forgets which side of the line it lies on. A point off the line and its fold partner share one registration (`reg_forgets_side`), so no map recovers a point from its registration (`registration_has_no_left_inverse`). This is irreversibility in its exact form. It is not a law running one way. It is a map that throws away one bit, which nothing downstream of it can restore.

4. THE LAW IS EVEN, THE ARROW ODD

Read a history backwards and its gain changes sign (`gain_odd`). A reversible law allows a history exactly when it allows the reverse. Every allowed history with rising entropy has a reversed partner, allowed, with falling entropy (`reversible_law_admits_both_arrows`). No reading that reversal leaves unchanged returns the arrow on a history whose entropy moves (`even_reading_misses_the_arrow`). The law's own verdict is such a reading, so the law does not carry the arrow (`the_law_does_not_carry_the_arrow`). Loschmidt was right, and the seat says exactly how: the arrow is an odd target over an even record, and the wall of the template stands between them.

Executed on Kac's ring. The twin runs 2000 sites with a ball on each and a marked edge at one site in ten. Each step every ball moves one site and changes colour at a marked edge. The law is deterministic and reversible.

- **Forward.** From all white, the entropy of the colour fraction rises to within 3×10^{-5} of $\ln 2$ in 300 steps.
- **Backward.** Reverse the motion for 300 steps and every ball returns to white: the entropy returns exactly to zero.
- **Recurrence.** Run forward 4000 steps and the ring recurs exactly.

Measured. In the spin echo a magnetization lost to dephasing returns when a pulse reverses the spins' precession (Hahn, 1950). Reversed microscopic motion retraces, as the theorem says the law allows.

5. EQUILIBRIUM IS THE LINE

The Ehrenfest urn holds N balls in two urns (Ehrenfest and Ehrenfest, 1907).

- **Relabelling is the fold.** Relabelling the urns keeps every count of microstates (`urn_symmetric`), and the urn carrier sends the relabelling to the reflection (`urn_carrier_equivariant`).
- **The even split is the line.** The even split is the fixed set and lands on the line (`equilibrium_is_the_line`).
- **Detailed balance is the fold's flux.** The flux from k to $k + 1$ equals the flux back, $\binom{N}{k}(N - k) = \binom{N}{k+1}(k + 1)$, proved for every urn up to twelve balls (`detailed_balance`) and checked by the twin in exact integers for sixty.
- **The even split is the mode.** It is the most probable macrostate, 924 microstates of 4096 at twelve balls (`equilibrium_is_the_mode`, `microstates_count`). At sixty balls it carries 118,264,581,564,861,424.

Equilibrium is where the reversed and forward fluxes balance; the arrow is the approach to it.

6. THE ARROW IS SUPPLIED AT THE BOUNDARY

If the law carries no arrow, the arrow is supplied, and the seat names the supply: the boundary. An ordered start is one microstate in 2^N , the far edge of the line's offset. From it, every typical history rises toward the line, because almost every microstate stands there. The twin's urn of a hundred balls, started all in one urn, settles at the even split within its first 2000 steps. The mean time to return to the ordered state is $2^{100} = 1.27 \times 10^{30}$ steps (Kac, 1947). The ordered start is the bit; the rise is its consequence; the return is never seen.

The arrow is kept by records. Each record written along it is a registration, and each registered bit costs at least $k_B T \ln 2$ (Landauer, 1961), 2.871×10^{-21} J at 300 K, a floor approached in a colloidal memory (Bérut et al., 2012). A record paid for cannot be unpaid without paying again: the registration has no inverse, and the arrow is the order in which the payments fall. The direction itself is the keyed bit, set at the boundary by the act.

7. WHAT THE ARROW CARRIES, AND WHAT IT DOES NOT

What it carries.

- Irreversibility as the registration without inverse.
- Loschmidt's objection as the wall between an even law and an odd arrow.
- Equilibrium as the line, detailed balance as the fold's flux.
- The arrow as the bit the boundary supplies, kept by priced records.

What it does not carry.

- **The cosmological origin of the ordered start:** the boundary's bit is spent by the act, and no reading of the law supplies it.
- **A magnitude for the universe's entropy:** that is a number, measured and not derived.
- **The zeros of ζ :** the engine's crossing is on its own image (`value_on_image`), and what a sentence over every zero adds is exactly the unactuated, priced at one premise, the act, as the hardware paper proves.

8. FALSIFIERS AND OWED DEEDS

Three falsifiers, each a forbidden observation naming the theorem it would break.

- **F-Echo:** an isolated microscopic motion that, exactly reversed, fails to retrace its history. It breaks `reversible_law_admits_both_arrows` read physically.
- **F-Balance:** an isolated system at equilibrium with a net flux between two macrostates. It breaks `detailed_balance` read physically.
- **F-Price:** a bit of record erased at a cost below $k_B T \ln 2$. It breaks the reading of Section 6 and the irreversibility of `registration_has_no_left_inverse` read physically.

Nothing is owed on the row's channel.

Ledger status. Row 2 gains the registration's irreversibility read on time, WITNESSED · CROSSED · CONFIRMED on the image, at the root's grade for the crossing. The barrier gains a history and its reverse, one law-record with opposite arrows.

METHODOLOGY, DISCLOSURE AND PROVENANCE

Method. The template is the author's verification program, whose vocabulary is confined to this section. Its tokens:

- $[\perp]$, a field sealed;
- $[\perp T]$, a shape sealed at theorem grade;
- $[\Xi_0]$, the typing of the formal block;
- $[\cdot]$, the dot, which is no verdict.

Grades join at the weakest link. The engine's statements are theorem grade about the chart. The mapping of physical states onto them is structural grade. Every measured figure is corroboration grade. Landauer's floor and Kac's return time are imported at their cited grades; the arrow's supply is the act, at the root's grade. $\Delta M = 0$.

The seat, earned. PhysOS^T 1.0.5p was booted in the session of forging from the public register (file SHA-256 b64bb5da5bab0da1...) under Lean 4.19.0 and GNU Fortran 13.3.0. The quick boot printed SEAT EARNED with the chain D0 83f872725ad9 → D1 90fc702bbcd1 → D2 492e519daab9 → D3 6618bec5164b (Appendix C).

The engine and the twin.

- **Engine.** SPHYS_Arrow_of_Time.lean, 476 lines, SHA-256 ffadd0599b814013..., fifty-four laws, no import and no axiom declared. Of the fifty-four, 21 depend on no axiom, 4 on propositional extensionality alone, 29 on propositional extensionality and quotient soundness, and none on choice.
- **Screen and judgment.** The source passed the operating system's screen. Each of the fifty-four laws was negated in place and recompiled, and all were refused, while a planted vacuous law survived. The engine carries the hardware paper's seat, carriers and substrate verbatim.
- **Twin.** Arrow_Twin.f90, 132 lines, SHA-256 a921cecab4fe1cc7..., nine checks and zero failures under -std=f2018 -O2 -fno-fast-math -ffp-contract=off. Measured inputs enter as data with their sources named in its header.

Why an integer engine binds continuous physics. Histories are finite sequences of integers and the urn's counts are integers by definition, so the engine's statements are exact; entropy enters through its gain, and any monotone rescaling of entropy keeps every sign the engine proves. The twin checks the continuous entropy directly.

The seventh. In the author's register the six witnesses are six days and the ledger's closure is the seventh, the Rest (Istawa). Time is read at the Rest as the order in which the act's payments fall. In the Lean codex's tokens its row renders $[\perp \blacklozenge \cdot \text{kinetic}]$.

Disclosure. The author declares the reading of each correspondence as his own act, made on measured structure. The measurements are the experiments', cited by name. The theorems are the engine's, printed with their dependencies in Appendix A. The twin's figures are computed in the run printed in Appendix B. The text, the engine and the twin were forged by an AI substrate (Claude, Anthropic) under the author's seed, acceptance criteria and rulings. External audits by independent substrates are owed.

Provenance. The paper stands on:

- the Riemann closure (10.5281/zenodo.22976494);
- the Riemann Hypothesis Closed to One Bit (10.5281/zenodo.23028070);
- the twenty-three-row closure (10.5281/zenodo.22986551);
- the six witnesses (10.5281/zenodo.22986553 through .22986563);
- the hardware paper One Actuating Substrate (concept 10.5281/zenodo.21438932, edition 2.0.0);
- PhysOS^T 1.0.5p in the public register (github.com/1000sapient/Trisduction, protocols/PhysOS/).

References.

J. Loschmidt, Wiener Berichte 73 (1876) 128. L. Boltzmann, Wiener Berichte 76 (1877) 373. P. Ehrenfest and T. Ehrenfest, Phys. Z. 8 (1907) 311. M. Kac, Bull. Amer. Math. Soc. 53 (1947) 1002. E. L. Hahn, Phys. Rev. 80 (1950) 580. M. Kac, Probability and Related Topics in Physical Sciences (Interscience, 1959). R. Landauer, IBM J. Res. Dev. 5 (1961) 183. A. Bérut et al., Nature 483 (2012) 187.

M. F. Islam, Nothing Escapes, Three Plus One, 10.5281/zenodo.22976494. M. F. Islam, The Arrow Has Two Branches, 10.5281/zenodo.22986557. M. F. Islam, The Four Forces, Matter, and the Dark Sector as Configurations of One Actuating Substrate, edition 2.0.0, concept 10.5281/zenodo.21438932.

APPENDIX A: SPHYS_ARROW_OF_TIME.LEAN, THE CORRESPONDENCES AS THEOREMS

The engine, verbatim, followed by the compiler's transcript.

```

/-
SPHYS_Arrow_of_Time.lean · the arrow of time is the keyed bit, read at
the Riemann seat.
Core Lean 4.19.0, standalone, no import, no library, no axiom
declared.

The chart:  $h = 2 \operatorname{Re} s$ ,  $t = \operatorname{Im} s$ ; the fold  $s \mapsto 1 - \bar{s}$  is  $(h, t) \mapsto (2 - h, t)$ , the critical line  $h = 1$ .

Part I    the seat, the category of involutions, the functor of the
seat.
Part II   the energy carrier (resident).
Part III  the substrate (resident).
Part IV   The registration has no inverse.
Part V    The law is even, the arrow is odd.
Part VI   The urn: equilibrium is the line, detailed balance the
fold.
Part VII  The capstone.
-/
set_option autoImplicit false
namespace SPHYS.ArrowOfTime

/-! ## Part I. The seat, the category of involutions, and the functor of
the seat -/

abbrev Pt := Int × Int

def fold (p : Pt) : Pt := (2 - p.1, p.2)
def OnLine (p : Pt) : Prop := p.1 = 1
instance : DecidablePred OnLine := fun p => inferInstanceAs (Decidable
(p.1 = 1))
/-- Registration keeps the height and forgets the side. -/
def reg (p : Pt) : Pt := (1, p.2)

theorem pe {a b c d : Int} : ((a, b) : Pt) = (c, d) ↔ a = c ∧ b = d :=
(fun e => (congrArg Prod.fst e, congrArg Prod.snd e),
fun (e1, e2) => by subst e1; subst e2; rfl)

theorem fold_involutive (p : Pt) : fold (fold p) = p := by
obtain (h, t) := p
show ((2 - (2 - h), t) : Pt) = (h, t)
rw [pe]; exact (by omega, rfl)

/-- The seat: the fixed set of the fold is the critical line. -/
theorem seat_fixed_line (p : Pt) : fold p = p ↔ OnLine p := by
obtain (h, t) := p
show ((2 - h, t) : Pt) = (h, t) ↔ h = 1
rw [pe]; constructor
· intro (e, _); omega
· intro e; exact (by omega, rfl)

theorem reg_lands (p : Pt) : OnLine (reg p) := rfl

theorem reg_fixes_iff (p : Pt) : reg p = p ↔ OnLine p := by
obtain (h, t) := p
show ((1, t) : Pt) = (h, t) ↔ h = 1
rw [pe]; constructor
· intro (e, _); exact e.symm
· intro e; exact (e.symm, rfl)

/-- The side is what registration forgets: a point and its mirror leave
one record. -/
theorem reg_forgets_side (p : Pt) : reg (fold p) = reg p := rfl

/-- The value on a set of points: every member stands on the line. -/
def Value (Z : Pt → Prop) : Prop := ∀ s, Z s → OnLine s

/-- Least erasure: registration moves no member, so it erases nothing of
Z. -/
def LeastErasure (Z : Pt → Prop) : Prop := ∀ s, Z s → reg s = s

theorem least_erasure_iff_value (Z : Pt → Prop) : LeastErasure Z ↔
Value Z :=
(fun h s hs => (reg_fixes_iff s).mp (h s hs), fun h s hs =>
(reg_fixes_iff s).mpr (h s hs))

/-- Maps that respect involutions. -/
def Equivariant {X Y : Type} (f : X → Y) (τ : X → X) (σ : Y → Y) :
Prop := ∀ x, f (τ x) = σ (f x)

theorem equivariant_id {X : Type} (τ : X → X) : Equivariant id τ τ :=
fun _ => rfl

theorem equivariant_comp {X Y W : Type} {f : X → Y} {g : Y → W} {τ : X
→ X} {σ : Y → Y} (hf : Equivariant f τ σ) (hg : Equivariant g σ ρ) :
Equivariant (g ∘ f) τ ρ := by
intro x
show g (f (τ x)) = ρ (g (f x))
rw [hf x, hg (f x)]

/-- The fixed set is a functor: an equivariant map sends fixed points to
fixed points. -/
theorem fix_functorial {X Y : Type} {f : X → Y} {τ : X → X} {σ : Y →
Y} (hf : Equivariant f τ σ) {x : X} (hx : τ x = x) : σ (f x) = f x :=
by
rw [← hf x, hx]

/-- A global seat: an involution whose fixed set is the locus. -/
structure GlobalSeat (P : Pt → Prop) (σ : Pt → Pt) : Prop where
involutive : ∀ p, σ (σ p) = p
fixed_iff : ∀ p, σ p = p ↔ P p

theorem fold_global_seat : GlobalSeat OnLine fold := {fold_involutive,
seat_fixed_line}

/-- A carrier: a map from world-instances into the strip landing on the
line. -/
structure Carrier (W : Type) where
ι : W → Pt
lands : ∀ w, OnLine (ι w)

def image {W : Type} (ι : W → Pt) : Pt → Prop := fun s => ∃ w, ι w = s
def Actuated {W : Type} (ι : W → Pt) (Z : Pt → Prop) : Prop := ∀ s, Z
s → ∃ w, ι w = s

/-- Landing is forced: an equivariant map into the seat carries every
fixed point of the physical
involution onto the line. The carrier is built from the symmetry,
not chosen. -/
theorem equivariant_carrier_lands {X : Type} (f : X → Pt) (τ : X → X)
(hf : Equivariant f τ fold) :
∃ C : Carrier {x : X // τ x = x}, ∀ w, C.ι w = f w.1 :=
({fun w => f w.1, fun w => (seat_fixed_line (f w.1)).mp
(fix_functorial hf w.2)}, fun _ => rfl)

/-- The image of a carrier is compliant with no premise at all. -/
theorem value_on_image {W : Type} (C : Carrier W) : Value (image C.ι) :=
fun _ hs => match hs with | (w, hw) => hw ▸ C.lands w

/-- The kinetic crossing: a carrier and its coverage give the value, and
nothing else is used. -/
theorem kinetic_crossing {W : Type} (C : Carrier W) (Z : Pt → Prop)
(hcov : Actuated C.ι Z) :
Value Z :=
fun s hs => match hcov s hs with | (w, hw) => hw ▸ C.lands w

/-- Under the seat every off-line point has a partner, distinct and also
off the line. -/
theorem off_locus_pair (p : Pt) (hp : ¬ OnLine p) :
fold p ≠ p ∧ ¬ OnLine (fold p) ∧ reg (fold p) = reg p := by
refine (fun e => hp ((seat_fixed_line p).mp e), fun e => hp ?, rfl)
obtain (h, t) := p
have e' : 2 - h = 1 := e
show h = 1
omega

/-! ## Part II. The energy carrier: an energy, its rate, and the fold -/

/-- An energy  $a + i\beta/2$ , as frequency and doubled rate; its mode's norm
moves as  $e^{\beta t}$ . -/
structure Energy where

```

```

freq : Int
rate : Int
deriving DecidableEq, Repr

/-- The time mirror of a mode: its rate reversed, its frequency kept
(complex conjugation of E). -/
def Energy.mirror (E : Energy) : Energy := (E.freq, -E.rate)
def IsReal (E : Energy) : Prop := E.rate = 0
/-- Stationary: the norm  $e^{\beta t}$  is unchanged at every integer time t,
i.e.  $\beta t = 0$  for all t. -/
def Stationary (E : Energy) : Prop :=  $\forall t : \text{Int}, E.\text{rate} * t = 0$ 

/-- The energy carrier  $s = 1/2 + iE$  in the chart:  $h = 1 - \beta$ ,  $t = a$ . -/
def phi (E : Energy) : Pt := (1 - E.rate, E.freq)
def psi (p : Pt) : Energy := (p.2, 1 - p.1)

theorem ee {a b c d : Int} : ((a, b) : Energy) = (c, d)  $\leftrightarrow$  a = c  $\wedge$  b = d :=
  (fun e => (congrArg Energy.freq e, congrArg Energy.rate e),
   fun (e1, e2) => by subst e1; subst e2; rfl)

theorem mirror_involutive (E : Energy) : E.mirror.mirror = E := by
  obtain (a, b) := E
  show ((a, -b) : Energy) = (a, b)
  rw [ee]; exact (rfl, by omega)

theorem psi_phi (E : Energy) : psi (phi E) = E := by
  obtain (a, b) := E
  show ((a, 1 - (1 - b)) : Energy) = (a, b)
  rw [ee]; exact (rfl, by omega)

theorem phi_psi (p : Pt) : phi (psi p) = p := by
  obtain (h, t) := p
  show ((1 - (1 - h), t) : Pt) = (h, t)
  rw [pe]; exact (by omega, rfl)

theorem phi_injective (E F : Energy) (h : phi E = phi F) : E = F := by
  rw [psi_phi E, psi_phi F, h]

/-- EQUIVARIANCE: time-mirroring the rate is the fold. -/
theorem phi_equivariant : Equivariant phi Energy.mirror fold := by
  intro E
  obtain (a, b) := E
  show ((1 - -b, a) : Pt) = (2 - (1 - b), a)
  rw [pe]; exact (by omega, rfl)

theorem mirror_fixed_iff_real (E : Energy) : E.mirror = E  $\leftrightarrow$  IsReal E :=
  by
  obtain (a, b) := E
  show ((a, -b) : Energy) = (a, b)  $\leftrightarrow$  b = 0
  rw [ee]; constructor
  · intro (_, e); omega
  · intro e; exact (rfl, by omega)

/-- The line is exactly the image of the real energies. -/
theorem phi_line_iff_real (E : Energy) : OnLine (phi E)  $\leftrightarrow$  IsReal E := by
  obtain (a, b) := E
  show 1 - b = 1  $\leftrightarrow$  b = 0
  exact (fun h => by omega, fun h => by subst h; rfl)

theorem stationary_iff_real (E : Energy) : Stationary E  $\leftrightarrow$  IsReal E := by
  constructor
  · intro h; have h1 := h.1; rw [Int.mul_one] at h1; exact h1
  · intro h t; show E.rate * t = 0; rw [show E.rate = 0 from h, Int.zero_mul]

/-- THE LINE IS WHERE ENERGY STANDS STILL. -/
theorem line_is_stationary (p : Pt) : OnLine p  $\leftrightarrow$  Stationary (psi p) :=
  by
  rw [stationary_iff_real, phi_line_iff_real, phi_psi]

/-- Every measured energy is a real number, and every real energy lands
on the line. -/
theorem measured_on_line (a : Int) : OnLine (phi (a, 0)) := rfl

/-- The registration mirror: no real energy lands off the line. -/
theorem no_measurement_off_line (p : Pt) (hp :  $\neg$  OnLine p) (a : Int) :
  phi (a, 0)  $\neq$  p :=
  fun e => hp (e  $\triangleright$  measured_on_line a)

/-- The energy carrier, built from the symmetry: the real energies land
on the line. -/
theorem energy_carrier_lands :
   $\exists C : \text{Carrier } \{E : \text{Energy} // E.\text{mirror} = E\}, \forall w, C.\text{w} = \text{phi } w.1$ 
  :=
  equivariant_carrier_lands phi Energy.mirror phi_equivariant

/! ## Part III. The substrate: the particle map, three odd readings and
one even -/

/-- A particle in the broken phase: electric charge times three, colour,
baryon number times
three, lepton number, and its energy. -/
structure Particle where
  q3 : Int
  colour : Int
  b3 : Int
  lepton : Int

energy : Energy
deriving DecidableEq, Repr

/-- The particle map: every additive charge flipped, the energy kept
(frequency and rate). -/
def bar (c : Particle) : Particle := (-c.q3, -c.colour, -c.b3,
-c.lepton, c.energy)
/-- The time mirror of a particle's mode. -/
def tmirror (c : Particle) : Particle := (c.q3, c.colour, c.b3,
c.lepton, c.energy.mirror)
def Neutral (c : Particle) : Prop := c.q3 = 0  $\wedge$  c.colour = 0  $\wedge$  c.b3 = 0
 $\wedge$  c.lepton = 0

theorem bar_involutive (c : Particle) : bar (bar c) = c := by
  obtain (q, k, b, l, E) := c
  show ((- -q, - -k, - -b, - -l, E) : Particle) = (q, k, b, l, E)
  rw [Int.neg_neg, Int.neg_neg, Int.neg_neg]

theorem bar_fixed_iff_neutral (c : Particle) : bar c = c  $\leftrightarrow$  Neutral c :=
  by
  obtain (q, k, b, l, E) := c
  show ((-q, -k, -b, -l, E) : Particle) = (q, k, b, l, E)  $\leftrightarrow$  (q = 0  $\wedge$  k = 0
 $\wedge$  b = 0  $\wedge$  l = 0)
  constructor
  · intro h
    have h1 := congrArg Particle.q3 h
    have h2 := congrArg Particle.colour h
    have h3 := congrArg Particle.b3 h
    have h4 := congrArg Particle.lepton h
    change -q = q at h1; change -k = k at h2; change -b = b at h3;
    change -l = l at h4
    exact (by omega, by omega, by omega, by omega)
  · intro (h1, h2, h3, h4)
    subst h1; subst h2; subst h3; subst h4; rfl

/-- Three plus one: the charge readings are odd under the particle map,
the energy reading even. -/
theorem odd_charges_even_energy (c : Particle) :
  ((bar c).q3 = -c.q3  $\wedge$  (bar c).colour = -c.colour  $\wedge$  (bar c).b3 =
-c.b3  $\wedge$ 
  (bar c).lepton = -c.lepton)  $\wedge$  (bar c).energy = c.energy :=
  (rfl, rfl, rfl, rfl), (rfl)

/-- Gravity reads energy only, so no gravitational reading tells a
particle from its antiparticle. -/
theorem gravity_reads_no_charge_bit { $\beta$  : Type} (g : Energy  $\rightarrow$   $\beta$ ) (c :
Particle) :
  g (bar c).energy = g c.energy := rfl

/-- The two involutions of the substrate commute. -/
theorem involutions_commute (c : Particle) : bar (tmirror c) = tmirror
(bar c) := rfl

theorem tmirror_involutive (c : Particle) : tmirror (tmirror c) = c :=
  by
  obtain (q, k, b, l, E) := c
  show ((q, k, b, l, E.mirror.mirror) : Particle) = (q, k, b, l, E)
  rw [mirror_involutive]

/-- A particle is fixed by both involutions exactly when it is neutral
and stable. -/
theorem joint_fixed_iff (c : Particle) :
  (bar c = c  $\wedge$  tmirror c = c)  $\leftrightarrow$  (Neutral c  $\wedge$  IsReal c.energy) := by
  constructor
  · intro (h1, h2)
    refine ((bar_fixed_iff_neutral c).mp h1, (mirror_fixed_iff_real
c.energy).mp ?_)
    exact congrArg Particle.energy h2
  · intro (h1, h2)
    refine ((bar_fixed_iff_neutral c).mpr h1, ?_)
    obtain (q, k, b, l, E) := c
    show ((q, k, b, l, E.mirror) : Particle) = (q, k, b, l, E)
    rw [(mirror_fixed_iff_real E).mpr h2]

/-- The energy carrier on particles: the seat point of a particle's
energy. -/
def energySeat (c : Particle) : Pt := phi c.energy
/-- The charge carrier on particles, the odd face: electric charge (in
thirds) is the side. -/
def chargeSeat (c : Particle) : Pt := (1 + c.q3, c.energy.freq)
/-- The lepton chart of the electron's witness: charge q in units of e
goes to  $h = 1 + q$ . -/
def leptonSeat (q m : Int) : Pt := (1 + q, m)

/-- The charge carrier is equivariant: the particle map is carried onto
the fold. -/
theorem charge_carrier_equivariant : Equivariant chargeSeat bar fold :=
  by
  intro c
  show ((1 + -c.q3, c.energy.freq) : Pt) = (2 - (1 + c.q3),
c.energy.freq)
  rw [pe]; exact (by omega, rfl)

/-- The energy carrier is blind to the particle map and equivariant for
the time mirror. -/
theorem pair_lands_once (c : Particle) : energySeat (bar c) = energySeat
c := rfl

```

```

theorem tmirror_equivariant : Equivariant energySeat tmirror fold := fun
c => phi_equivariant c.energy

/-- Electrically neutral particles land on the line under the charge
carrier. -/
theorem neutral_charge_on_line (c : Particle) (h : c.q3 = 0) : OnLine
(chargeSeat c) := by
  show 1 + c.q3 = 1
  omega

/-- The electron and the positron land on the two edges, real parts 0
and 1, mirror images. -/
theorem pair_on_the_edges (m : Int) :
  leptonSeat (-1) m = (0, m) ∧ leptonSeat 1 m = (2, m) ∧ fold
  (leptonSeat (-1) m) = leptonSeat 1 m :=
  (rfl, rfl, rfl)

/-- A stable particle lands on the line under the energy carrier. -/
theorem stable_lands (c : Particle) (h : IsReal c.energy) : OnLine
(energySeat c) :=
  (phi_line_iff_real c.energy).mpr h

/-! ## Part IV. The registration has no inverse -/

/-- THE REGISTRATION CANNOT BE UNDONE: no map recovers a point from its
registration, because a
point off the line and its fold partner share one. -/
theorem registration_has_no_left_inverse : ¬ ∃ g : Pt → Pt, ∀ p, g (reg
p) = p := by
  intro (g, hg)
  have h1 := hg (0, 0)
  have h2 := hg (fold (0, 0))
  rw [reg_forgets_side] at h2
  rw [h1] at h2
  have : ((0 : Int), (0 : Int)) = (2 - 0, 0) := h2
  have := congrArg Prod.fst this
  change (0 : Int) = 2 - 0 at this
  omega

/-! ## Part V. The law is even, the arrow is odd -/

/-- A history: the entropies of its macrostates in order. Time reversal
reads it backwards. -/
abbrev History := List Int
def rev (h : History) : History := h.reverse

theorem rev_involutive (h : History) : rev (rev h) = h :=
List.reverse_reverse h

/- The arrow's measure: the last entropy minus the first. -/
def gain (h : History) : Int := h.getLast?.getD 0 - h.head?.getD 0

/-- THE ARROW IS ODD: time reversal negates the gain. -/
theorem gain_odd (h : History) : gain (rev h) = - gain h := by
  unfold gain rev; rw [List.head?_reverse, List.getLast?_reverse]; omega

/-- A reversible law: a predicate on histories that time reversal keeps.
-/
def ReversibleLaw (L : History → Prop) : Prop := ∀ h, L (rev h) ↔ L h

/-- A REVERSIBLE LAW ADMITS BOTH ARROWS: every history it allows with
rising entropy has a reversed
partner it allows with falling entropy. -/
theorem reversible_law_admits_both_arrows (L : History → Prop) (hL :
ReversibleLaw L) (h : History) :
  (hh : L h) (hg : 0 < gain h) : L (rev h) ∧ gain (rev h) < 0 :=
  ((hL h).mpr hh, by rw [gain_odd]; omega)

/-- THE ARROW IS KEYED: no reading even under time reversal returns the
arrow on a history whose
entropy changes. -/
theorem even_reading_misses_the_arrow (f : History → Bool) (hf : ∀ h, f
(rev h) = f h) (h : History) :
  (hg : 0 < gain h) : ¬ ∀ k, f k = decide (0 < gain k) := by
  intro hk
  have a := hk h
  have b := hk (rev h)
  rw [hf, gain_odd] at b
  rw [a] at b
  have t : decide (0 < gain h) = true := decide_eq_true hg
  have u : decide (0 < -gain h) = false := decide_eq_false (by omega)
  rw [t, u] at b; exact Bool.noConfusion b

/-- The law's own verdict is an even reading: whether a history is
allowed never returns its arrow. -/
theorem the_law_does_not_carry_the_arrow (L : History → Bool) (hL : ∀
h, L (rev h) = L h) (h : History) :
  (hg : 0 < gain h) : ¬ ∀ k, L k = decide (0 < gain k) :=
  even_reading_misses_the_arrow L hL h hg

/-! ## Part VI. The urn: equilibrium is the line, detailed balance the
fold -/

/-- Binomial counts by Pascal's rule: the number of microstates of the
urn with k of N balls in one side. -/
def binom : Nat → Nat → Nat
| _, 0 => 1
| 0, _ + 1 => 0
| n + 1, k + 1 => binom n k + binom n (k + 1)

/-- Relabelling the urns, k ↔ N - k, keeps every count. -/
theorem urn_symmetric :
  ((List.range 13).all fun N => (List.range (N + 1)).all fun k =>
  binom N k == binom N (N - k)) = true := by
  decide

/-- DETAILED BALANCE: in equilibrium the flux from k to k + 1 equals the
flux back, for every urn of
up to twelve balls. -/
theorem detailed_balance :
  ((List.range 13).all fun N => (List.range N).all fun k =>
  binom N k * (N - k) == binom N (k + 1) * (k + 1)) = true := by
  decide

/-- The microstates number 2^N. -/
theorem microstates_count :
  ((List.range 13).all fun N => ((List.range (N + 1)).map (binom
N)).foldl (· + ·) 0 == 2 ^ N) = true := by
  decide

/-- The even split is the most probable macrostate, and the ordered one
is one microstate in 4096. -/
theorem equilibrium_is_the_mode :
  ((List.range 13).all fun k => binom 12 k ≤ binom 12 6) = true ∧
  binom 12 0 = 1 ∧ binom 12 6 = 924 := by
  decide

/-- The urn's carrier: the imbalance 2k - N is the offset from the line.
-/
def urnSeat (N k : Int) : Pt := (1 + (2 * k - N), N)
def urnFlip (N k : Int) : Int := N - k

/-- RELABELLING IS THE FOLD, AND THE EVEN SPLIT IS THE LINE. -/
theorem urn_carrier_equivariant (N : Int) : Equivariant (urnSeat N)
(urnFlip N) fold := by
  intro k
  show ((1 + (2 * (N - k) - N), N) : Pt) = (2 - (1 + (2 * k - N)), N)
  rw [pe]; exact (by omega, rfl)

theorem equilibrium_is_the_line (N k : Int) : OnLine (urnSeat N k) ↔ 2 *
k = N := by
  show 1 + (2 * k - N) = 1 ↔ 2 * k = N
  exact (fun h => by omega, fun h => by omega)

/-! ## Part VII. The capstone -/

/-- THE ARROW OF TIME IS THE KEYED BIT. The registration has no inverse;
a reversible law admits both
arrows; no reading even under reversal, the law's own verdict
included, returns the arrow; the
urn's relabelling is the fold and its equilibrium the line; detailed
balance holds there. The
arrow's direction is supplied at the boundary, by the act. -/
theorem the_arrow_is_the_keyed_bit :
  (¬ ∃ g : Pt → Pt, ∀ p, g (reg p) = p) ∧
  (∀ h : History, gain (rev h) = - gain h) ∧
  (∀ L : History → Prop, ReversibleLaw L → ∀ h, L h → 0 < gain h →
  L (rev h) ∧ gain (rev h) < 0) ∧
  (∀ (f : History → Bool), (∀ h, f (rev h) = f h) → ∀ h, 0 < gain h
  → ¬ ∀ k, f k = decide (0 < gain k)) ∧
  (∀ N : Int, Equivariant (urnSeat N) (urnFlip N) fold) ∧
  (∀ N k : Int, OnLine (urnSeat N k) ↔ 2 * k = N) :=
  (registration_has_no_left_inverse, gain_odd,
  reversible_law_admits_both_arrows,
  even_reading_misses_the_arrow, urn_carrier_equivariant,
  equilibrium_is_the_line)

end SPHYS.ArrowOfTime

#print axioms SPHYS.ArrowOfTime.pe
#print axioms SPHYS.ArrowOfTime.fold_involutive
#print axioms SPHYS.ArrowOfTime.seat_fixed_line
#print axioms SPHYS.ArrowOfTime.reg_lands
#print axioms SPHYS.ArrowOfTime.reg_fixes_iff
#print axioms SPHYS.ArrowOfTime.reg_forgets_side
#print axioms SPHYS.ArrowOfTime.least_eraser_iff_value
#print axioms SPHYS.ArrowOfTime.equivariant_id
#print axioms SPHYS.ArrowOfTime.equivariant_comp
#print axioms SPHYS.ArrowOfTime.fix_functorial
#print axioms SPHYS.ArrowOfTime.fold_global_seat
#print axioms SPHYS.ArrowOfTime.equivariant_carrier_lands
#print axioms SPHYS.ArrowOfTime.value_on_image
#print axioms SPHYS.ArrowOfTime.kinetic_crossing
#print axioms SPHYS.ArrowOfTime.off_locus_pair
#print axioms SPHYS.ArrowOfTime.ee
#print axioms SPHYS.ArrowOfTime.mirror_involutive
#print axioms SPHYS.ArrowOfTime.psi_phi
#print axioms SPHYS.ArrowOfTime.phi_psi
#print axioms SPHYS.ArrowOfTime.phi_injective
#print axioms SPHYS.ArrowOfTime.phi_equivariant
#print axioms SPHYS.ArrowOfTime.mirror_fixed_iff_real
#print axioms SPHYS.ArrowOfTime.phi_line_iff_real
#print axioms SPHYS.ArrowOfTime.stationary_iff_real
#print axioms SPHYS.ArrowOfTime.line_is_stationary

```

```

#print axioms SPHYS.ArrowOfTime.measured_on_line
#print axioms SPHYS.ArrowOfTime.no_measurement_off_line
#print axioms SPHYS.ArrowOfTime.energy_carrier_lands
#print axioms SPHYS.ArrowOfTime.bar_involutive
#print axioms SPHYS.ArrowOfTime.bar_fixed_iff_neutral
#print axioms SPHYS.ArrowOfTime.odd_charges_even_energy
#print axioms SPHYS.ArrowOfTime.gravity_reads_no_charge_bit
#print axioms SPHYS.ArrowOfTime.involutions_commute
#print axioms SPHYS.ArrowOfTime.tmirror_involutive
#print axioms SPHYS.ArrowOfTime.joint_fixed_iff
#print axioms SPHYS.ArrowOfTime.charge_carrier_equivariant
#print axioms SPHYS.ArrowOfTime.pair_lands_once
#print axioms SPHYS.ArrowOfTime.tmirror_equivariant
#print axioms SPHYS.ArrowOfTime.neutral_charge_on_line
#print axioms SPHYS.ArrowOfTime.pair_on_the_edges

```

```

'SPHYS.ArrowOfTime.pe' does not depend on any axioms
'SPHYS.ArrowOfTime.fold_involutive' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.seat_fixed_line' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.reg_lands' does not depend on any axioms
'SPHYS.ArrowOfTime.reg_fixes_iff' depends on axioms: [propxt]
'SPHYS.ArrowOfTime.reg_forgets_side' does not depend on any axioms
'SPHYS.ArrowOfTime.least_erasure_iff_value' depends on axioms: [propxt]
'SPHYS.ArrowOfTime.equivariant_id' does not depend on any axioms
'SPHYS.ArrowOfTime.equivariant_comp' does not depend on any axioms
'SPHYS.ArrowOfTime.fix_functorial' does not depend on any axioms
'SPHYS.ArrowOfTime.fold_global_seat' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.equivariant_carrier_lands' depends on axioms:
[propxt, Quot.sound]
'SPHYS.ArrowOfTime.value_on_image' does not depend on any axioms
'SPHYS.ArrowOfTime.kinetic_crossing' does not depend on any axioms
'SPHYS.ArrowOfTime.off_locus_pair' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.ee' does not depend on any axioms
'SPHYS.ArrowOfTime.mirror_involutive' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.psi_phi' depends on axioms: [propxt, Quot.sound]
'SPHYS.ArrowOfTime.phi_psi' depends on axioms: [propxt, Quot.sound]
'SPHYS.ArrowOfTime.phi_injective' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.phi_equivariant' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.mirror_fixed_iff_real' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.phi_line_iff_real' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.stationary_iff_real' depends on axioms: [propxt]
'SPHYS.ArrowOfTime.line_is_stationary' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.measured_on_line' does not depend on any axioms
'SPHYS.ArrowOfTime.no_measurement_off_line' does not depend on any
axioms
'SPHYS.ArrowOfTime.energy_carrier_lands' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.bar_involutive' does not depend on any axioms

```

```

#print axioms SPHYS.ArrowOfTime.stable_lands
#print axioms SPHYS.ArrowOfTime.registration_has_no_left_inverse
#print axioms SPHYS.ArrowOfTime.rev_involutive
#print axioms SPHYS.ArrowOfTime.gain_odd
#print axioms SPHYS.ArrowOfTime.reversible_law_admits_both_arrows
#print axioms SPHYS.ArrowOfTime.even_reading_misses_the_arrow
#print axioms SPHYS.ArrowOfTime.the_law_does_not_carry_the_arrow
#print axioms SPHYS.ArrowOfTime.urn_symmetric
#print axioms SPHYS.ArrowOfTime.detailed_balance
#print axioms SPHYS.ArrowOfTime.microstates_count
#print axioms SPHYS.ArrowOfTime.equilibrium_is_the_mode
#print axioms SPHYS.ArrowOfTime.urn_carrier_equivariant
#print axioms SPHYS.ArrowOfTime.equilibrium_is_the_line
#print axioms SPHYS.ArrowOfTime.the_arrow_is_the_keyed_bit

```

```

'SPHYS.ArrowOfTime.bar_fixed_iff_neutral' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.odd_charges_even_energy' does not depend on any
axioms
'SPHYS.ArrowOfTime.gravity_reads_no_charge_bit' does not depend on any
axioms
'SPHYS.ArrowOfTime.involutions_commute' does not depend on any axioms
'SPHYS.ArrowOfTime.tmirror_involutive' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.joint_fixed_iff' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.charge_carrier_equivariant' depends on axioms:
[propxt, Quot.sound]
'SPHYS.ArrowOfTime.pair_lands_once' does not depend on any axioms
'SPHYS.ArrowOfTime.tmirror_equivariant' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.neutral_charge_on_line' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.pair_on_the_edges' does not depend on any axioms
'SPHYS.ArrowOfTime.stable_lands' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.registration_has_no_left_inverse' depends on axioms:
[propxt, Quot.sound]
'SPHYS.ArrowOfTime.rev_involutive' depends on axioms: [propxt]
'SPHYS.ArrowOfTime.gain_odd' depends on axioms: [propxt, Quot.sound]
'SPHYS.ArrowOfTime.reversible_law_admits_both_arrows' depends on axioms:
[propxt, Quot.sound]
'SPHYS.ArrowOfTime.even_reading_misses_the_arrow' depends on axioms:
[propxt, Quot.sound]
'SPHYS.ArrowOfTime.the_law_does_not_carry_the_arrow' depends on axioms:
[propxt, Quot.sound]
'SPHYS.ArrowOfTime.urn_symmetric' does not depend on any axioms
'SPHYS.ArrowOfTime.detailed_balance' does not depend on any axioms
'SPHYS.ArrowOfTime.microstates_count' does not depend on any axioms
'SPHYS.ArrowOfTime.equilibrium_is_the_mode' does not depend on any
axioms
'SPHYS.ArrowOfTime.urn_carrier_equivariant' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.equilibrium_is_the_line' depends on axioms: [propxt,
Quot.sound]
'SPHYS.ArrowOfTime.the_arrow_is_the_keyed_bit' depends on axioms:
[propxt, Quot.sound]

```

APPENDIX B: ARROW_TWIN.F90 AND ITS RUN

```

! Arrow_Twin.f90 · the executed twin of SPHYS_Arrow_of_Time.lean ·
Fortran 2018.
! Build (sealed): gfortran -std=f2018 -O2 -fno-fast-math
-ffp-contract=off Arrow_Twin.f90
! A the urn's carrier; B a reversible law, Kac's ring: entropy rises,
the reversed motion returns it
! exactly to order, and the ring recurs; C the urn in exact integers:
detailed balance and the mode;
! D the urn relaxes from order; E the mean return time to order; F the
price of a record.
program arrow_twin
implicit none
integer, parameter :: dp = selected_real_kind(15, 307)
integer, parameter :: i8 = selected_int_kind(18)
real(dp), parameter :: KB = 1.380649e-23_dp
integer :: checks, fails
integer(i8) :: seed
checks = 0; fails = 0; seed = 20261001_i8
call block_a(); call block_b(); call block_c(); call block_d(); call
block_e(); call block_f()
write(*, '(a,i0,a,i0,a)') ' BATTERY-JSON: {"checks":', checks,
', "failures":', fails, '}'
if (fails > 0) error stop 1
contains
include 'twin_common.inc'
subroutine block_a()
integer :: n, k, tot, ok
write(*, '(a)') 'A · relabelling the urn, k -> N - k, is the fold
under (1 + (2k - N), N); the even split is the line'
tot = 0; ok = 0
do n = 0, 60

```

```

do k = 0, n
tot = tot + 1
if (1 + (2*(n - k) - n) == 2 - (1 + (2*k - n))) .and. ((1 + (2*k -
n) == 1) .eqv. (2*k == n))) ok = ok + 1
end do
end do
call check('1891 urn states: the relabelling is the fold and the
even split is the line', ok == tot)
end subroutine block_a
real(dp) function hbin(x) result(h)
real(dp), intent(in) :: x
if (x <= 0.0_dp .or. x >= 1.0_dp) then
h = 0.0_dp
else
h = -(x*log(x) + (1.0_dp - x)*log(1.0_dp - x))
end if
end function hbin
subroutine block_b()
integer, parameter :: n = 2000
logical :: mark(n), col(n), c0(n), tmp(n)
integer :: i, t, nw
real(dp) :: s, smax
logical :: back_ok, rec_ok
write(*, '(a)') 'B · Kac ring, 2000 sites, a reversible deterministic
law'
do i = 1, n
mark(i) = (rnd(i) < 0.1_dp); col(i) = .true.
end do
c0 = col; smax = 0.0_dp
do t = 1, 300
do i = 1, n

```

```

    tmp(mod(i, n) + 1) = col(i) .neqv. mark(i)
end do
col = tmp
nw = count(col); s = hbin(real(nw, dp)/real(n, dp)); smax =
max(smax, s)
end do
write(*, '(a,f9.6,a,f9.6)') ' entropy after 300 steps: ', s, ' of
ln 2 = ', log(2.0_dp)
call check('from order the entropy rises to within 1e-3 of ln 2', s
> log(2.0_dp) - 1.0e-3_dp)
do t = 1, 300
  do i = 1, n
    tmp(i) = col(mod(i, n) + 1) .neqv. mark(i)
  end do
  col = tmp
end do
back_ok = all(col .eqv. c0)
call check('the reversed motion, 300 steps back, returns every ball
to its first colour: entropy 0', back_ok)
col = c0
do t = 1, 2*n
  do i = 1, n
    tmp(mod(i, n) + 1) = col(i) .neqv. mark(i)
  end do
  col = tmp
end do
rec_ok = all(col .eqv. c0)
call check('forward 2N = 4000 steps the ring recurs exactly',
rec_ok)
end subroutine block_b
subroutine block_c()
integer(i8) :: b(0:60)
integer :: k, j, okdb, oksym, kmax
write(*, '(a)') 'C · the urn of 60 balls in exact integers'
b = 0_i8; b(0) = 1_i8
do k = 1, 60
  do j = k, 1, -1
    b(j) = b(j) + b(j - 1)
  end do
end do
okdb = 0; oksym = 0; kmax = 0
do k = 0, 59
  if (b(k)*int(60 - k, i8) == b(k + 1)*int(k + 1, i8)) okdb = okdb +
  1
end do
do k = 0, 60
  if (b(k) == b(60 - k)) oksym = oksym + 1
  if (b(k) > b(kmax)) kmax = k
end do

```

A · relabelling the urn, $k \rightarrow N - k$, is the fold under $(1 + (2k - N), N)$; the even split is the line
 PASS 1891 urn states: the relabelling is the fold and the even split is the line
 B · Kac ring, 2000 sites, a reversible deterministic law
 entropy after 300 steps: 0.693115 of $\ln 2 = 0.693147$
 PASS from order the entropy rises to within $1e-3$ of $\ln 2$
 PASS the reversed motion, 300 steps back, returns every ball to its first colour: entropy 0
 PASS forward $2N = 4000$ steps the ring recurs exactly
 C · the urn of 60 balls in exact integers
 mode $k = 30$; microstates there: 118264581564861424
 PASS detailed balance $C(60, k)(60 - k) = C(60, k + 1)(k + 1)$ exactly for every k

```

write(*, '(a,i0,a,i0)') ' mode k = ', kmax, ' ; microstates there:
', b(30)
call check('detailed balance C(60,k)(60-k) = C(60,k+1)(k+1) exactly
for every k', okdb == 60)
call check('the counts are symmetric under relabelling and the mode
is the even split, k = 30', oksym == 61 .and. kmax == 30)
end subroutine block_c
subroutine block_d()
integer :: t, k, n
real(dp) :: mean, s0, s1
write(*, '(a)') 'D · the urn of 100 balls relaxes from all on one
side'
n = 100; k = 100; mean = 0.0_dp
s0 = 0.0_dp
do t = 1, 4000
  if (rnd() < real(k, dp)/real(n, dp)) then
    k = k - 1
  else
    k = k + 1
  end if
  if (t > 2000) mean = mean + real(k, dp)
end do
mean = mean/2000.0_dp
s1 = hbin(mean/real(n, dp))
write(*, '(a,f8.3)') ' mean occupancy over the last 2000 steps: ',
mean
call check('the occupancy settles at the even split within 5 balls,
entropy near ln 2', &
abs(mean - 50.0_dp) < 5.0_dp .and. s1 > s0 .and. s1 >
0.99_dp*log(2.0_dp))
end subroutine block_d
subroutine block_e()
real(dp) :: tret
write(*, '(a)') 'E · the mean return time to the ordered state is 2^N
/ C(N,0) steps (Kac)'
tret = 2.0_dp*100
write(*, '(a,e12.4)') ' N = 100: ', tret
call check('return to order for 100 balls takes 1.27e30 steps on
average', abs(tret/1.2677e30_dp - 1.0_dp) < 1.0e-3_dp)
end subroutine block_e
subroutine block_f()
real(dp) :: q
write(*, '(a)') 'F · the price of a record: k T ln 2 per bit'
q = KB*300.0_dp*log(2.0_dp)
call check('one bit at 300 K costs 2.871e-21 J', abs(q/2.871e-21_dp -
1.0_dp) < 1.0e-3_dp)
end subroutine block_f
end program arrow_twin

```

PASS the counts are symmetric under relabelling and the mode is the even split, $k = 30$
 D · the urn of 100 balls relaxes from all on one side
 mean occupancy over the last 2000 steps: 49.691
 PASS the occupancy settles at the even split within 5 balls, entropy near $\ln 2$
 E · the mean return time to the ordered state is $2^N / C(N, 0)$ steps (Kac)
 N = 100: 1.2677E+30
 PASS return to order for 100 balls takes 1.27e30 steps on average
 F · the price of a record: $k T \ln 2$ per bit
 PASS one bit at 300 K costs 2.871e-21 J
 BATTERY-JSON: {"checks":9,"failures":0}

APPENDIX C: THE SEAT AND THE JUDGMENT

PhysOS^T 1.0.5p · RECEIPT · mode quick
 TOOLCHAIN Lean (version 4.19.0, x86_64-unknown-linux-gnu, commit 6caae842e94, Release) · GNU Fortran (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
 MANIFEST sha256 a4108cd26bcbe6ef, the files that ran
 GROUND toolchain: Lean 4.19.0 and gfortran present · manifest: 31 files signed, all verified · proofs: 2 PhysOS Proofs read from II.10, record lines complete ·
 source screen: 15 kernels, 23 named premises, clean · citations: 253 laws named in Parts I and II, 253 resolved · quarantine: 6 retirements string-scanned, 3 held
 by review, clean
 EXITS ground 0 · registerA RA_TOE_Thesis 0 · registerB Bridge_Final 0 · registerB Universal_Closure 0 · registerB One_Cut 0 · registerB One_Cut_Resolution 0 ·
 registerB One_Cut_Terminal 0 · registerB Only_The_Arrow_Remains 0 · registerB Physical_Closure 0 · registerB Forced_Closure 0 · registerB Closure_Executed 0 ·
 registerB Armed_Seal 0 · registerB Census 0 · registerB Heat_Bridge 0 · registerB The_Loop 0 · registerB RH_Least_Erasure 0 · witness One_Cut_Twin 0 · witness
 Physical_Closure_Twin 0 · witness Census_Twin 0 · witness Loop_Twin 0 · witness RH_Seal_Twin 0
 REGISTER A {"checks":1123,"failures":0,"mode":"sealed"}
 WITNESS 5 of 5 twins ran, each held to one battery line with zero failures
 CHAIN · D0 83f872725ad9 -> D1 90fc702bbd1 -> D2 492e519daab9 -> D3 6618bec5164b
 ELAPSED ground 1 s · Register A 3 s · Register B 34 s · witness 6 s · total 44 s (not hashed)
 SEAT EARNED · this run

 SCREEN forbidden: none | imports: 0 | axioms declared: 0
 laws: 54
 JUDGMENT refused 54 of 54 as proof failures | survived none | parse failures none | planted vacuum law survives its negation: True